

Makefile cheatsheet

变量赋值

```
1  foo = "bar"
2  bar = $(foo) foo # 动态（递归）赋值
3  foo := "boo"      # 一次性赋值, $(bar) 现在变为 "boo foo"
4  foo ?= /usr/local # 安全赋值,如果前面已经被赋值过就不会执行, $(foo) and $(bar) still the
                        same
5  bar += world      # 追加赋值, $(bar) 现在是 "boo foo world"
6  foo != echo fooo  # 执行shell command 并且赋值给 foo 变量
7  # $(bar) now is "fooo foo world"
```

注意！ = 仅在使用时计算

自动变量（Magic变量）

```
1  out.o: src.c src.h
2      $@ # "out.o" (目标)
3      $< # "src.c" (第一个依赖)
4      $^ # "src.c src.h" (所有依赖, 去重)
5
6  %.o: %.c
7      $* # 模式匹配中的词干 stem ("foo.c" 中匹配到的 "foo")
8
9  also:
10     $+ # prerequisites (所有依赖, 不去重)
11     $? # prerequisites (新的依赖)
12     $| # prerequisites (order-only 依赖)
13
14     $(@D) # target directory
```

order-only 依赖是什么？去 STFW

命令前缀

参数	含义
-	忽略错误
@	不要打印命令
+	即使 Make 处于'don't execute'模式下，依旧执行

```
1 build:
2     @echo "compiling"
3     -gcc $< $@
4
5 -include .depend
```

查找文件

```
1 js_files := $(wildcard test/*.js)
2 all_files := $(shell find images -name "*")
```

替换

```
1 file      = $(SOURCE:.cpp=.o)    # foo.cpp => foo.o
2 outputs   = $(files:src/%.coffee=lib/%.js)
3
4 outputs   = $(patsubst %.c, %.o, $(wildcard *.c)) # .c能匹配的文件都替换为.o
5 assets    = $(patsubst images/%, assets/%, $(wildcard images/*))
```

更多常用函数

```
1 $(strip $(string_var)) # 去空格
2
3 $(filter %.less, $(files)) # 过滤出.less文件
4 $(filter-out %.less, $(files)) # 过滤掉 .less文件
```

Includes

```
1 -include foo.make
```

make选项

```
1 make
2 -e, --允许环境变量覆盖 makefile 中定义的变量
3 -B, --强制重新构建所有目标, 无论文件时间戳如何
4 -s, --silent
5 -j, --jobs=N    # 并行处理
```

条件命令

```
1  foo: $(objects)
2  ifeq ($(CC),gcc)
3      $(CC) -o foo $(objects) $(libs_for_gcc)
4  else
5      $(CC) -o foo $(objects) $(normal_libs)
6  endif
```

递归处理

```
1  deploy:
2      $(MAKE) deploy2
```